



PROCESO DE DESARROLLO DE SOFTWARE

Grupo 13
Trabajo Práctico Obligatorio

Integrantes:

Tomas Szkarlatiuk, 1190913

Delia Tomas, 1188972

Rocha Germán, 1195326

Profesora:

LEON MARIA ANGELA

Fase 1	4
Análisis del problema y diseño inicial	4
Sistema de Gestión Hotelera y Servicios para Huéspedes	4
1. Descripción general de la problemática	5
2. Objetivos del sistema	5
Objetivo general	5
Objetivos específicos	5
3. Alcance funcional	6
Fuera del alcance	6
4. Requisitos funcionales	7
RF1	7
RF2	7
RF3	7
RF4	7
RF5	7
RF6	7
RF7	7
RF8	7
RF9	7
RF10	7
RF11	8
RF12	8
5. Requisitos no funcionales	8
RNF1	8
RNF2	8
RNF3	8
RNF4	8
RNF5	8
RNF6	8
RNF7	8
RNF8	8
RNF9	9
6. Diagrama de clases	9
Clases principales candidatas	9
Relaciones principales	10
7. Identificación de actores o usuarios	10
Administrador	10
Recepcionista	11
Huésped	11
8. Casos de uso principales	11
CU1 — Registrar huésped	11
CU2 — Registrar habitación	11
CU3 — Crear reserva	11
CU4 — Consultar disponibilidad	11

CU5 — Confirmar reserva	11
CU6 — Cancelar reserva	11
CU7 — Agregar servicio adicional	11
CU8 — Calcular costo final	11
CU9 — Aplicar promoción	12
CU10 — Registrar pago	12
CU11 — Consultar historial	12
9. Diagrama de casos de uso	12
10. Primer listado de clases candidatas	13
Clases de dominio	13
Clases de control	14
- ReservaController	14
- HabitacionController	14
- HuespedController	14
- PagoController	14
Clases de persistencia	14
- ReservaRepository	14
- HabitacionRepository	14
- HuespedRepository	14
- PagoRepository	14
Interfaces candidatas	14
Clases futuras para patrones	14
11. Justificación preliminar de patrones de diseño	14
HabitacionFactory — Factory Method	14
ServicioDecorator — Decorator	15
ReservaState — State	15
DescuentoStrategy — Strategy	15
SistemaHotelFacade — Facade	15
DatabaseConnection — Singleton	15
12. Consideraciones iniciales de diseño	15
13. Repositorio Git	16
Estructura aproximada	16
Distribución preliminar de tareas	16
Integrante 1	16
Integrante 2	16
Integrante 3	17
14. Conclusión	17

FASE 1

- Análisis del problema y diseño inicial

Sistema de Gestión Hotelera y Servicios para Huéspedes

1. Descripción general de la problemática

Una cadena hotelera que administra distintos establecimientos turísticos necesita modernizar y centralizar la gestión de reservas, habitaciones, huéspedes, servicios adicionales y procesos administrativos asociados a la estadía de los clientes.

Actualmente, gran parte de la administración se realiza mediante sistemas separados, registros manuales y herramientas poco integradas, lo que provoca:

- errores en reservas,
- problemas de disponibilidad,
- dificultades para administrar promociones,
- demoras operativas,
- poca flexibilidad para incorporar nuevos servicios,
- inconsistencias en la información,
- dificultades para mantener la trazabilidad de las operaciones.

La organización busca desarrollar un sistema orientado a objetos que permita administrar de forma eficiente las operaciones hoteleras y que pueda adaptarse fácilmente a nuevas necesidades comerciales y tecnológicas.

Además, la empresa desea incorporar persistencia de datos y almacenamiento en base de datos para mantener la información de huéspedes, reservas, habitaciones y servicios de manera organizada y segura.

El sistema deberá permitir gestionar reservas, habitaciones, huéspedes, servicios adicionales, estados de estadía y promociones, manteniendo una arquitectura flexible, reutilizable y extensible.

2. Objetivos del sistema

Objetivo general

- Desarrollar un sistema de gestión hotelera que permita centralizar y optimizar la administración de reservas, habitaciones y servicios para huéspedes mediante una solución orientada a objetos con persistencia de datos.
-

Objetivos específicos

- Registrar huéspedes y habitaciones.
 - Gestionar reservas y estadías.
 - Consultar disponibilidad de habitaciones.
 - Permitir agregar servicios adicionales.
 - Calcular costos de hospedaje.
 - Administrar estados de reservas.
 - Registrar y almacenar información en base de datos.
 - Facilitar futuras ampliaciones del sistema.
 - Aplicar patrones de diseño, principios SOLID y GRASP.
 - Mantener una arquitectura desacoplada y organizada.
-

3. Alcance funcional

El sistema permitirá:

- Registrar huéspedes.
 - Registrar habitaciones.
 - Crear reservas.
 - Confirmar o cancelar reservas.
 - Consultar disponibilidad.
 - Gestionar servicios adicionales.
 - Gestionar promociones y descuentos.
 - Calcular costos finales.
 - Administrar estados de reservas.
 - Generar información básica de estadías.
 - Persistir información en base de datos.
 - Consultar historial de reservas.
-

Fuera del alcance

En esta primera versión NO se incluirán:

- pagos online,

- aplicación web,
- integración real con APIs externas,
- autenticación avanzada,
- aplicación móvil,
- interfaz gráfica avanzada.

La ejecución será realizada mediante consola Java utilizando persistencia en base de datos.

4. Requisitos funcionales

RF1

- El sistema deberá permitir registrar huéspedes.

RF2

- El sistema deberá permitir registrar habitaciones.

RF3

- El sistema deberá permitir crear reservas.

RF4

- El sistema deberá permitir consultar la disponibilidad de habitaciones.

RF5

- El sistema deberá permitir confirmar o cancelar reservas.

RF6

- El sistema deberá permitir agregar servicios adicionales a una estadia.

RF7

- El sistema deberá calcular el costo total de la reserva.

RF8

- El sistema deberá permitir administrar distintos estados de reserva.

RF9

- El sistema deberá mostrar información de huéspedes, habitaciones y reservas.

RF10

- El sistema deberá almacenar información en base de datos.

RF11

- El sistema deberá permitir consultar historial de reservas.

RF12

- El sistema deberá permitir aplicar promociones o descuentos.
-

5. Requisitos no funcionales**RNF1**

- El sistema deberá estar desarrollado en Java.

RNF2

- El sistema deberá aplicar programación orientada a objetos.

RNF3

- El código deberá mantener bajo acoplamiento y alta cohesión.

RNF4

- La arquitectura deberá permitir futuras ampliaciones.

RNF5

- El sistema deberá poseer una estructura clara de paquetes.

RNF6

- El sistema deberá incluir documentación UML coherente.

RNF7

- El sistema deberá permitir la reutilización de componentes.

RNF8

- El sistema deberá utilizar persistencia mediante base de datos relacional.

RNF9

- El sistema deberá mantener separadas las capas de lógica, persistencia y dominio.
-

6. Diagrama de clases**Clases principales candidatas**

Huesped

- id
- nombre
- dni
- email
- telefono

Habitacion

- numero
- tipo
- precioBase
- disponible

Reserva

- fechaIngreso
- fechaSalida
- estado
- costoTotal

ServicioAdicional

- nombre
- precio

Promocion

- nombre
- porcentajeDescuento

Hotel

- nombre
- habitaciones
- reservas

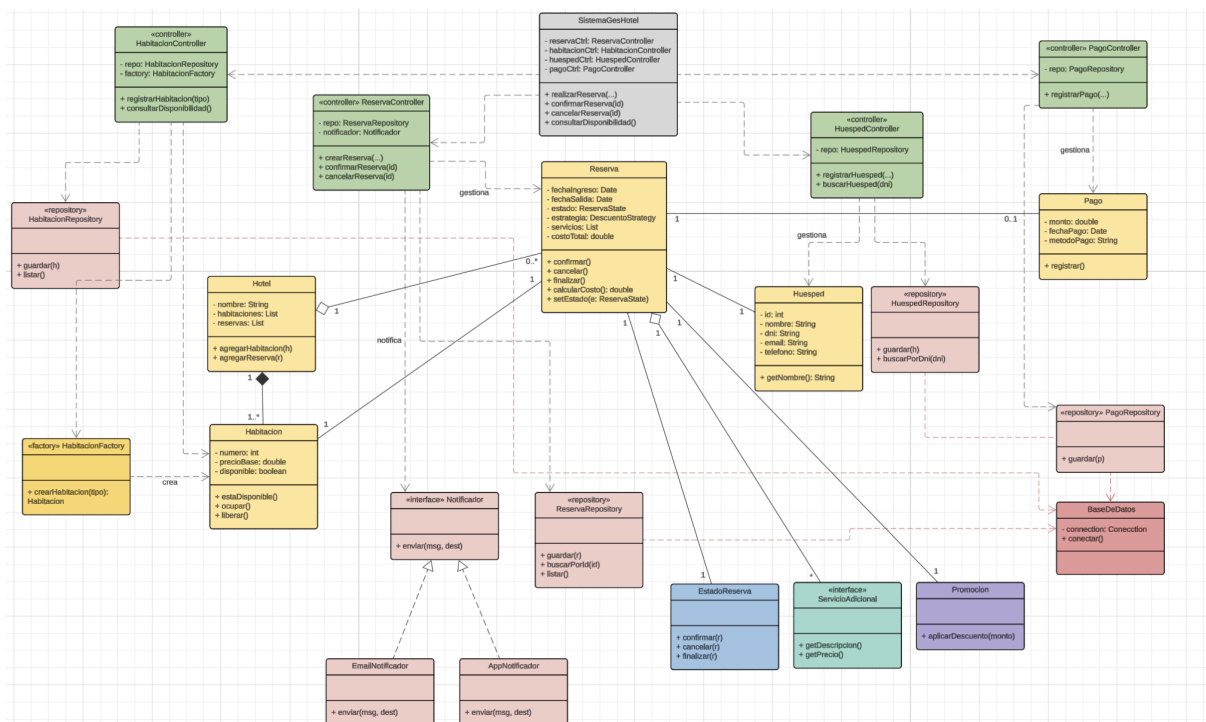
Pago

- monto
- fechaPago
- metodoPago

Notificador

CalculadorPrecio

DatabaseConnection



Relaciones principales

Hotel ----- Habitacion

Hotel ----- Reserva

Reserva ----- Huesped

Reserva ----- Habitacion

Reserva ----- ServicioAdicional

Reserva ----- Promocion

Reserva ----- Pago

7. Identificación de actores o usuarios

Administrador

- Responsable de administrar habitaciones, reservas, huéspedes y promociones.

Recepcionista

- Encargado de registrar reservas y gestionar estadias.

Huésped

- Cliente que realiza reservas y utiliza servicios del hotel.
-

8. Casos de uso principales

CU1 — Registrar huésped

- El recepcionista registra un nuevo huésped en el sistema.

CU2 — Registrar habitación

- El administrador registra habitaciones disponibles.

CU3 — Crear reserva

- El recepcionista crea una reserva para un huésped.

CU4 — Consultar disponibilidad

- El sistema muestra habitaciones disponibles.

CU5 — Confirmar reserva

- El recepcionista confirma una reserva pendiente.

CU6 — Cancelar reserva

- El recepcionista cancela una reserva existente.

CU7 — Agregar servicio adicional

- Se agregan servicios a la estadía del huésped.

CU8 — Calcular costo final

- El sistema calcula el costo total de la estadía.

CU9 — Aplicar promoción

- El sistema aplica descuentos o promociones sobre reservas.

CU10 — Registrar pago

- El sistema registra pagos asociados a una reserva.

CU11 — Consultar historial

- El sistema permite consultar reservas anteriores.
-

9. Diagrama de casos de uso

Administrador

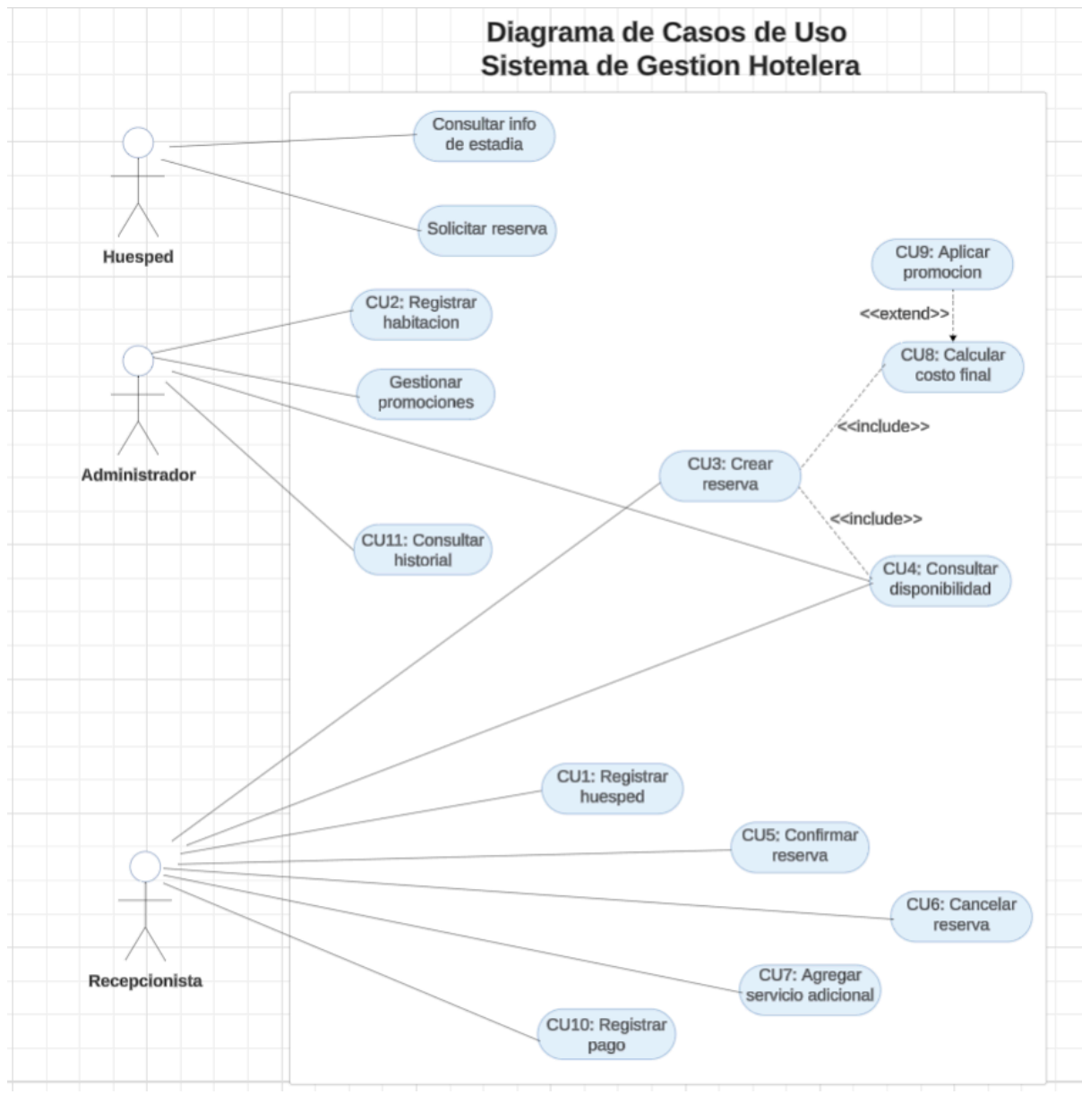
- Registrar habitación
- Consultar reservas
- Administrar habitaciones
- Gestionar promociones
- Consultar historial

Recepcionista

- Registrar huésped
- Crear reserva
- Confirmar reserva
- Cancelar reserva
- Consultar disponibilidad
- Agregar servicios
- Registrar pagos

Huésped

- Solicitar reserva
- Consultar información de estadía



10. Primer listado de clases candidatas

Clases de dominio

- Huesped
- Habitacion
- Reserva
- Hotel
- ServicioAdicional
- Promocion
- Pago

Clases de control

- ReservaController
 - HabitacionController
 - HuespedController
 - PagoController
-

Clases de persistencia

- ReservaRepository
 - HabitacionRepository
 - HuespedRepository
 - PagoRepository
-

Interfaces candidatas

- Notificador
 - CalculadorPrecio
-

Clases futuras para patrones

- HabitacionFactory
 - ReservaState
 - DescuentoStrategy
 - SistemaHotelFacade
 - DatabaseConnection
 - EmailNotificador
 - AppNotificador
-

11. Justificación preliminar de patrones de diseño

HabitacionFactory — Factory Method

- Se utilizará el patrón Factory Method para centralizar la creación de habitaciones según su tipo (Standard, Premium o Suite), evitando dependencias directas con clases concretas y facilitando futuras ampliaciones del sistema.

ServicioDecorator — Decorator

- Se aplicará el patrón Decorator para permitir agregar servicios adicionales a una reserva o habitación de forma dinámica, como desayuno, spa o cochera, sin modificar la estructura principal de las clases.

ReservaState — State

- Se utilizará el patrón State para administrar los distintos estados de una reserva (Pendiente, Confirmada, Cancelada y Finalizada), permitiendo cambiar comportamientos según el estado actual.

DescuentoStrategy — Strategy

- Se aplicará el patrón Strategy para implementar distintas políticas de cálculo de precios y promociones, permitiendo cambiar algoritmos de descuento sin modificar la lógica principal de reservas.

SistemaHotelFacade — Facade

- Se utilizará el patrón Facade para simplificar el acceso a los distintos subsistemas del hotel, centralizando operaciones relacionadas con reservas, huéspedes, habitaciones y servicios.

DatabaseConnection — Singleton

- Se aplicará el patrón Singleton para garantizar una única instancia de conexión y configuración de acceso a la base de datos durante la ejecución del sistema.

12. Consideraciones iniciales de diseño

Durante el desarrollo del sistema se buscará mantener:

- bajo acoplamiento,
- alta cohesión,
- correcta asignación de responsabilidades, siguiendo principios SOLID y patrones GRASP vistos en la materia.

Por ejemplo:

- se aplicará el principio de Responsabilidad Única (SRP) separando lógica de negocio, persistencia y controladores;
 - se utilizará Experto en Información asignando responsabilidades a las clases que poseen los datos necesarios;
 - se buscará mantener bajo acoplamiento entre módulos del sistema para facilitar futuras ampliaciones y mantenimiento.
-

13. Repositorio Git

Estructura aproximada

ProyectoHotel

src

- model
- repository
- service
- controller
- strategy
- factory
- state
- facade
- utils
- main

Documentacion

- README.md
- .gitignore

Repositorio GitHub del Proyecto

<https://github.com/131310100505/SistemaGestionHoteleria>

Distribución preliminar de tareas

Integrante 1

- Análisis de requisitos funcionales y no funcionales.

- Elaboración de casos de uso.
- Desarrollo de diagramas UML.
- Documentación general del proyecto.

Integrante 2

- Desarrollo de clases de dominio.
- Implementación de relaciones entre clases.
- Organización de paquetes y estructura del proyecto.
- Implementación de funcionalidades principales del sistema.

Integrante 3

- Investigación y aplicación de patrones de diseño.
 - Aplicación y justificación de principios SOLID y GRASP.
 - Implementación de controladores y persistencia.
 - Gestión del repositorio Git y control de versiones.
-

14. Conclusión

La solución propuesta permitirá mejorar la gestión operativa hotelera mediante una arquitectura orientada a objetos flexible y extensible.

El sistema se encuentra pensado para facilitar futuras ampliaciones, incorporar persistencia mediante base de datos y permitir la correcta aplicación de patrones de diseño, principios SOLID y GRASP, manteniendo coherencia entre análisis, diseño e implementación.